

#UNIT-3

Date |

Page |

Expert Systems -

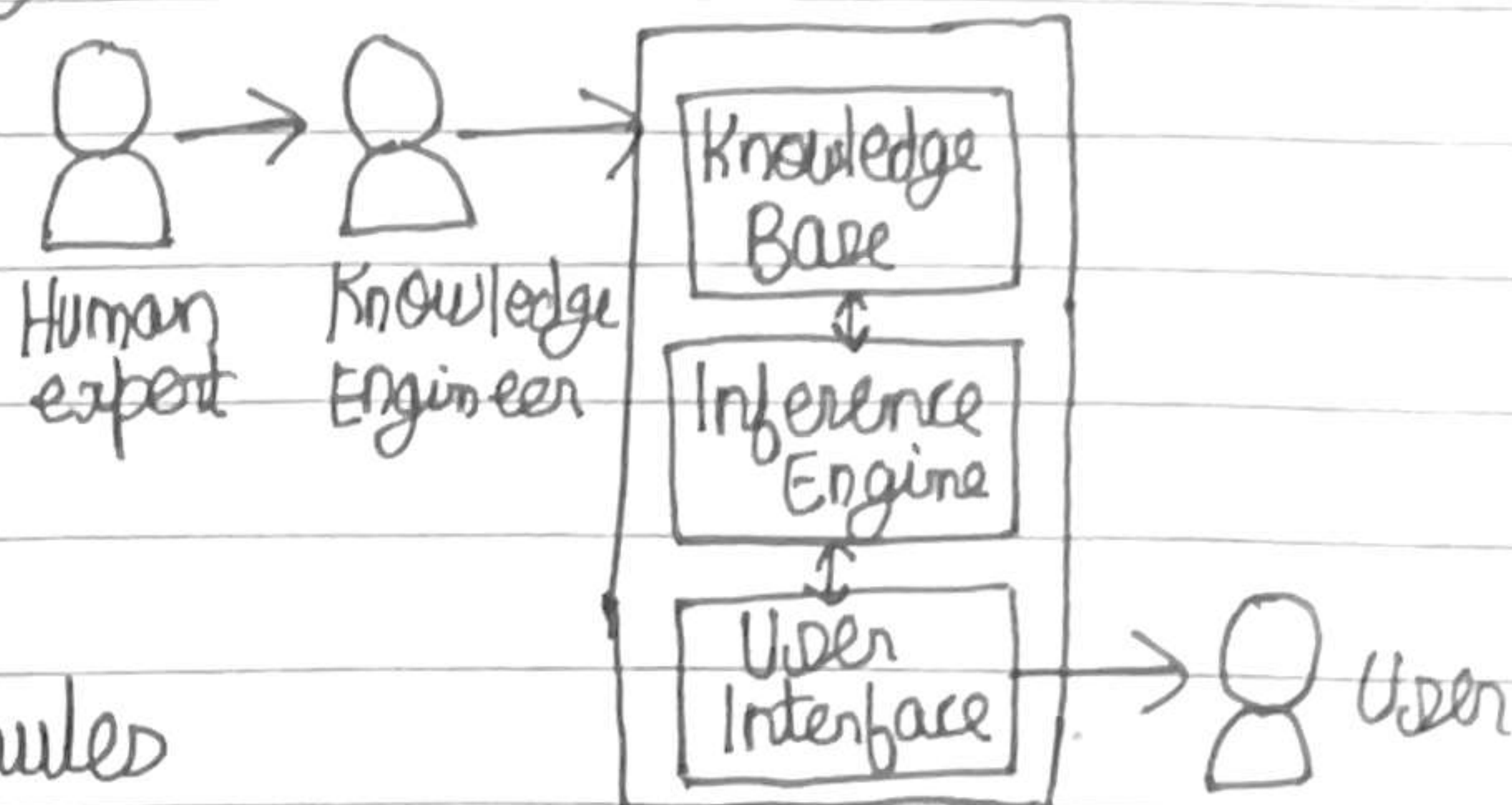
- AI system that mimics human expert decision making
- Uses Knowledge + inference rules
- Applied in medical, finance, engineering, etc
- eg- MYCIN (medical diagnosis)
- Applications:

- Healthcare
- Finance
- Manufacturing

- Structure:

- Knowledge base: Facts + rules
- Inference engine: Applied logic to derive conclusions
- User Interface: Interaction with user
- Explanation facility: Explains reasoning
- Knowledge Acquisition: Updates/improves knowledge

- Combined stored expertise + reasoning → faster, accurate decisions



User Interface -

- Medium for user interaction (input & output)
- Ensures clear, understandable results
- Features:

- User friendly navigation
- Clear result presentation
- Supports additional inputs

Inference Engine -

- Core component → applies rules on knowledge base
- Used:

- Forward chaining (facts → conclusion)
- Backward chaining (goal → facts)
- Handles conflicts by rule selection/prioritization

Knowledge Base + Inference Engine + UI $\rightarrow$ Expert Decision System

Date |

Page |

Knowledge Base -

- Stores facts, rules, relationship

- Structure:

 - IF-THEN rules, frames, ontologies

- Needs continuous updates (maintenance)

Expert System Life Cycle -

1) Identification: define problem/domain

2) Conceptualization: identify knowledge sources

3) Formalization: convert knowledge to formal form

4) Implementation: build system

5) Testing: validate with data

6) Deployment: release system

7) Maintenance: update & improve system

Limitations of Expert Systems -

1) Lack of common sense

2) High development cost

3) Limited learning capabilities

4) Dependency on knowledge base

Applications (Additional) -

1) DENDRAL: Analyzes chemical compounds

2) MYCIN: Diagnoses bacterial infections

3) PROSPECTOR: Used in geological exploration

4) XCON(R1): Configures computer systems at digital equipment Corporation

5) CLIPS: Tool for building expert systems

Knowledge acquisition in expert systems -

- Process of extracting, structuring, organizing knowledge
- Sources: experts / data / systems
- Types:
 - Manual: Direct from experts
 - Automated: ML-based
 - Semi automated: Hybrid

Types of Expert Systems -

- 1) Rule-Based: IF-THEN rules
- 2) Frame-Based: Used structured data (frames)
- 3) Fuzzy systems: Handles uncertainty (degrees of truth)
- 4) Neural-Based: Combine ANN + expert logic
- 5) Hybrid: Mix of multiple AI techniques

1) Rule-Based -

- Most common type
- Uses IF-THEN rules
- Works best in structured problems

2) Frame-Based -

- Uses frames (data structure)
- Stores attributes + values
- Suitable for complex relationships

3) Fuzzy Systems -

- Uses fuzzy logic (not just true/false)
- Handles uncertain/imprecise data
- Gives more flexible decisions

4) Neural Network-Based -

- Combines learning (ANN) + rules
- Improves performance over time
- Used for prediction & pattern recognition

5) Neuro Fuzzy -

- Combines neural networks + fuzzy logic
- Provides accuracy + adaptability
- Useful in complex real world problems

Planning = Search + actions + goal satisfaction

Date |

Page |

Planning in AI -

- Planning: Generates sequence of actions from initial $\rightarrow$ goal state
- Considers constraints
- Core idea: State A $\rightarrow$ Actions $\rightarrow$ State B
- Applications:
 - Robot navigation
 - Manufacturing automation
 - Game AI
 - Scheduling & logistics
 - Autonomous vehicles
- Elements:
 - Initial state: starting condition
 - Goal state: target condition
 - Actions (Operators):
 - Preconditions
 - Effects
 - State Space: All reachable states
 - Eg: Initial: Robot at A
Goal: Robot at C
Actions: Move(A,B), Move(B,C)

Classical Planning -

- | | |
|---|--|
| - Assumptions: <ul style="list-style-type: none">- Deterministic- Fully observable- Static environment- Single agent | - Representation: <ul style="list-style-type: none">- STRIPS- Predicate logic |
| | - Action model: <ul style="list-style-type: none">- Preconditions + Effects |

State-Space Planning -

(a) Forward (Progression):

- Starts from initial ~~node~~ state
- Apply actions forward
- Stops at goal
- Pro = Simple
- Con = Large search space

(b) Backward (Regression):

- Start from goal
- Work backward to initial
- Stops at initial
- Pro = Focused Search
- Con = Complex reasoning

Planning Algorithms -

1) STRIPS -

- Uses preconditions + effects
- Simple representation
- Used in classical planning

2) A* -

- Heuristic based optimal planning
- eg- distance to goal

3) Graph Plan -

- Builds planning graph
- Alternates: state levels & action levels
- Pro: efficient, shortest plans

4) SATplan -

- Converts planning $\rightarrow$ SAT problem
- Solved using SAT solvers

STRIPS -

- Classical AI planning algorithm
- Used: States, Actions, Goals
- Finds action sequence: Initial $\rightarrow$ Goal

Assumptions:

- Deterministic
- Fully observable
- Static
- Single agent

Actions Model:

Each action has:

- Preconditions (must be true before)
- Add List (facts to add)
- Delete List (facts to remove)

Key Points -

- (i) Uses preconditions + Add + Delete
- (ii) Based on state transitions
- (iii) Uses goal regression

- State updated by:

- Remove DELETE list
- Add ADD list

- STRIPS representation -

Planning Problem: $P = (S_0, G, A, S)$

- S_0 : Initial state

- G : Goal state

- A : Actions

- S : State space

States = set of predicates

Closed World assumption: missing facts = false

Eg -

Action: Move(x, y)

Preconditions: At(x), clear(y)

Add: At(y)

Delete: At(x)

- Advantages -

- Simple & intuitive

- Clear separation (knowledge vs reasoning)

- Basis for modern planning

- Good for classical planning

- Limitations -

- No uncertainty handling

- No time/resources support

- Not for dynamic environments

- Large (exponential) search space

- Applications -

- Robot planning

- Manufacturing

- Puzzle solving

- Game AI

- Algorithms -

- STRIPS - Action based planning

- A* - Optimal path using heuristics

- GraphPlan - Planning graph construction

- SATPlan - Convert planning $\rightarrow$ SAT problem

- Approaches -

- State-space (Search over states)

- Plan-space (search over plans)

- Hierarchical (high level $\rightarrow$ detailed steps)

- Challenges -

- Uncertainty

- Dynamic environment

- Incomplete information

Perception → Detection → Planning → Safe Navigation

Date |

Page |

Hierarchical Planning -

- Break tasks → sub tasks
- Pros: Reduces complexity
- eg - Travel → Book ticket
→ Pack → Board train

Multi-agent Planning -

- Multiple agents cooperate / complete
- Pros: Scalable, robust, adaptive
- eg - Swarm robotics traffic systems,
Multi robot systems

AI in Robotics & Computer Vision -

Why AI in robotics

- Enables perception + decision + action
- Computer vision → understand visual data
- Integrates sensing, perception, decision making

Applications -

- Autonomous vehicles
- Drones
- Industrial robots
- Medical / service robots

Core Components (Pipeline) -

Perception → Localization → Mapping
Control ← Planning ←

1) Perception -

- Sense & interpret environment
- Used: Cameras, LiDAR, Ultrasonic, Infrared
- Tasks: detect lanes, objects, signs

2) Localization -

- Determine robot position
- Methods
 - GPS
 - Odometry (wheel / visual)
 - SLAM

3) Mapping -

- Create environment representation
- Types
 - 2D grid
 - 3D point cloud
 - Semantic maps
- Helps in navigation & coverage

4) Planning -

- Decide optimal path/actions

- Types

- Path planning

- Motion planning

- Task planning

- Goal: shortest, safest route

5) Control -

- (converts plans $\rightarrow$ motor actions)

- Methods

- PID control

- Model Predictive Control (MPC)

- Adaptive Control

- Ensures stability & safety

Path Planning -

- Finds optimal path (start $\rightarrow$ goal)

- Considers: obstacles, distances, time, energy

- Types

- Global Planning $\rightarrow$ full map known

- Local Planning $\rightarrow$ real time, dynamic environment

- Algorithms -

(a) Classical

- Dijkstra $\rightarrow$ shortest path

- A* $\rightarrow$ optimal + heuristic

- RRT $\rightarrow$ random exploration

(b) AI-based

- Reinforcement Learning

- Neural network planners

Cost Function (A*) -

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost from start

$h(n)$ = heuristic to goal

Obstacle Avoidance -

- Basics -

- Real time detect + avoid obstacles
- Ensures safe navigation

- Techniques -

- Potential Field
- Vector Field Histogram (VFH)
- Dynamic Window Approach (DWA)

- Sensors -

- Cameras
- LiDAR
- Ultrasonic sensors

Object Detection & Recognition -

- Object Detection → find objects + location
- Object Recognition → classify objects
- Applications

- Face recognition
- Traffic sign detection
- Industrial inspection
- Surveillance systems

OpenCV -

- Open source computer vision library
- Used for image & video processing
- Enables real time analysis (see, detect, understand)
- Multi Language: Python, C++, Java
- Key characteristics
 - Free & widely used (BSD license)
 - Large developer community
 - Continuously updated
 - Optimized for real time performance
- Core Functionalities

(a) Image Processing

- Filtering (blur, edge detection)
- Color conversion (RGB → HSV)
- Threshold & segmentation

(b) Video Analysis

- Motion detection
- Object tracking
- Background subtraction

(c) Feature Detection

- Edge detection (Canny)
- Corner detection (Harris)
- Keypoints (SIFT, ORB)

OpenCV in AI & Robotics -

- Acts as robust vision systems
- Uses:

- Object detection & recognition
- Lane detection
- Face detection
- Gesture recognition
- Visual SLAM

- Processing pipeline:

- (i) Image acquisition
- (ii) Pre-processing
- (iii) Feature extraction
- (iv) Object detection/recognition
- (v) Output & decision making

- Advantages:

- Fast & efficient
- Cross platform
- Rich documentation
- Integrates with AI frameworks (TensorFlow, PyTorch)

- Limitations:

- Less accurate than deep learning
- Limited built-in deep learning models
- Needs tuning for complex scenarios

SLAM (Simultaneous Localization and Mapping) -

- Build map + estimate position simultaneously
- Used in unknown environments
- Essential when GPS unavailable/unreliable
- Purpose:
 - Enable navigation, path planning, obstacle avoidance
 - Helps robot make real time decisions

- Core Idea:

Robot:

- Maps environment
- Localizes itself in that map (same time)

- Key Challenges :

- Localization depends on map
- Map depends on localization
- circular dependency problem

- Key Components :

(a) Localization

- Estimate robot's position/orientation
- Uses landmarks, sensor data

(b) Mapping

- Create environment representation
- Enables navigation & planning

(c) Sensor Fusion

- Combines multiple sensors:
 - Cameras
 - LiDAR
 - IMU
- Improves accuracy & robustness

- Applications

- Autonomous vehicles
- Indoor robots (warehouse, vacuum)
- Drones
- Exploration (underwater / space)